



University of the Philippines Cebu

The Role of Number Theory in Password Hashing

Baba, Ashley B.
Maloloy-on, Irenia Vince D.
Merales, Benedicto M.

June 3, 2025

1 Introduction

Web applications are the most popular means of providing services and gathering information for many institutions. Their accessibility and ease of use have led to their widespread adoption across various sectors, including education, healthcare, banking, and government services. Because of their wide distribution and critical role in handling sensitive data, web applications have become an attractive target for attackers who seek to exploit vulnerabilities for malicious purposes such as identity theft, financial fraud, or unauthorized data access [4].

Most computer systems use passwords to ensure data security. A password is a user-remembered secret made up of typeable characters [5]. When creating an account, users are usually asked to come up with a password that is both secure and easy to remember. Many systems force users to create a complex password, which is composed of numbers, alphabets, and sometimes special characters, to prevent conventional password attacking methods. As a result, users often reuse the same password across multiple accounts, increasing the risk if one of them is compromised.

To address these concerns, many research studies have focused on enhancing password storage security techniques. The most popular technique is the use of hash functions. A hashing algorithm (HA) is a mathematical function that distorts data and makes it unreadable [6]. HAs are one-way programs, so the input cannot be deciphered by anyone else. Hashing always protects the input data, so even if someone gains access to the storage where the hash values are stored, the input data will remain secure. Hashing is also used for digital signatures and when indexing data, showing its versatility and importance in modern security systems.

Despite the widespread use of hashing algorithms to protect passwords, certain challenges remain. The security of these algorithms depends heavily on their ability to resist attacks that attempt to reverse or find collisions in the hash output. Simple hash functions can be vulnerable to such attacks, which may allow attackers to recover the original password or find two different inputs that produce the same hash [7]. These risks are especially significant in contexts where users frequently reuse passwords or where large volumes of password data are stored. In such cases, even a single weakness in the hashing process can lead to extensive breaches. As computational power increases and attack methods become more sophisticated, the design of secure and resilient hashing algorithms becomes not only more complex but also more critical to maintaining overall system security.

This is where number theory plays a crucial role in the design and security of modern hash functions. Number theory provides the mathematical foundation that makes modern hash functions secure and reliable. Concepts such as modular arithmetic, prime numbers, and one-way functions help create hash algorithms that are difficult to invert and resistant to collisions [8]. By applying these fundamental principles, cryptographers develop hashing techniques that significantly strengthen password security, making it far more difficult for attackers to access sensitive information. Furthermore, advances in number theory continue to inspire the development of new cryptographic primitives that improve the efficiency and security of password protection methods.

2 Fundamental Concepts

This section reviews essential concepts from number theory, which form the foundation for password hashing techniques.

Theorem 1. *The Division Algorithm.* *If a and b are integers such that $b > 0$, then there exist unique integers q and r such that*

$$a = bq + r \quad \text{with} \quad 0 \leq r < b.$$

Theorem 2. *The Euclidean Algorithm.* *Let $r_0 = a$ and $r_1 = b$ be nonnegative integers with $b \neq 0$. If the division algorithm is successively applied to obtain $r_j = r_{j+1}q_{j+1} + r_{j+2}$ with $0 < r_{j+2} < r_{j+1}$ for $j = 0, 1, 2, \dots, n-2$ and $r_n = 0$, then $(a, b) = r_{n-1}$, the last nonzero remainder.*

Theorem 3. *Fundamental Theorem of Arithmetic.* *Every positive integer $n > 1$ can be written uniquely as a product of primes, with the prime factors in the product written in order of nondecreasing size.*

Definition 1. *Congruence.* *If a and b are integers, we say that a is congruent to b modulo m if $m \mid (a - b)$.*

Theorem 4. *If a, b, c , and m are integers with $m > 0$ such that $a \equiv b \pmod{m}$, then*

1. $a + c \equiv b + c \pmod{m}$,
2. $a - c \equiv b - c \pmod{m}$,
3. $ac \equiv bc \pmod{m}$.

Theorem 5. *Let a, b , and m be integers with $m > 0$ and $\gcd(a, m) = d$. If $d \nmid b$, then the congruence $ax \equiv b \pmod{m}$ has no solutions. If $d \mid b$, then the congruence $ax \equiv b \pmod{m}$ has exactly d incongruent solutions modulo m .*

Theorem 6. *Chinese Remainder Theorem.* *Let $m_1, m_2, \dots, m_r$ be pairwise relatively prime positive integers. Then the system of congruences*

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ \vdots \\ x \equiv a_2 \pmod{m_2} \\ x \equiv a_r \pmod{m_r} \end{cases}$$

has a unique solution modulo $M = m_1 m_2 \cdots m_r$.

Definition 2. *Prime.* *An integer $a > 1$ is prime if the only positive divisors of a are 1 and a itself.*

Theorem 7. *Fermat's Little Theorem.* *If p is prime and a is a positive integer with $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$.*

Theorem 8. *Euler's Theorem.* *If m is a positive integer and a is an integer with $\gcd(a, m) = 1$, then $a^{\varphi(m)} \equiv 1 \pmod{m}$.*

Definition 3. *Primitive Root.* *If r and n are relatively prime integers with $n > 0$ and if $\text{ord}_n(r) = \varphi(n)$, then r is called a primitive root modulo n .*

Definition 4. *Modular Inverse.* *Given two integers a and m , the modular inverse of a modulo m is a number x such that $ax \equiv 1 \pmod{m}$.*

3 Number Theory in Password Hashing

Number Theory serves as the foundation for password hashing. Concepts like modular arithmetic, primes, and discrete logarithms allow developers to create a model for a hash.

3.1 Modular Arithmetic in Hashing

Modular arithmetic forms the backbone of many cryptographic and hashing algorithms [1]. It keeps numbers within a fixed range, creating predictable, bounded results that are essential for security and integrity.

Using Definition 1, developers can create an algorithm, say

$$H = aP + b \bmod p,$$

where a, b, p are integers. With this formula, they can:

1. keep outputs within a finite field.
2. enhance non-linearity, critical for the avalanche effect (where a small input change drastically changes the output)

With this hash function, however, it is easy to reverse the hash using modular inverse. Thus, developers use more complex functions of modular arithmetic.

Example 1. Let

$$H = \sum_{i=1}^n f(x_i) \bmod x_1$$

be the hash function where $f(x)$ is the function that returns the ASCII value of the letter. Say we hash "abc". The corresponding ASCII values are 97, 98, and 99. Thus by the functions, we have $H \equiv 97 + 98 + 99 \pmod{97} \equiv 3$. Thus, the hash code is 3. This is too simple of a hash code, and very inefficient since the order of the letters do not matter.

3.2 Primes and Cryptographic Strength

Primes, by Definition 2, are numbers where its divisors are only 1 and itself. Prime numbers are central in number theory and cryptography due to their properties. They create cyclic groups with good structure. Their use in modulus arithmetic ensures better randomness and unpredictability [2].

In hash functions and cryptosystems, they prevent patterns or repetitions in hash outputs. Algorithms like SHA-256 use prime-based constants to obscure relationships between input and output.

Example 2. Let us improve our hash function from Example 1 using primes. Let

$$H_{n-i+1} = f(x_i) \cdot p_{i+1} \bmod p_i$$

be the hash function where $f(x)$ is the function that returns the ASCII value of the letter, n is the number of characters to be hashed, and p_i is the prime of that index, ($p_1 = 2$, $p_2 = 3$, ...). Since there are no patterns as to which what prime goes next, reversing the hash would prove difficult. We also hashed each letter in reverse, so the arrangement would matter, unlike the previous example. Say we hash "cat". We have,

$$\begin{aligned}
H_3 &\equiv (116 \cdot 3) \bmod 2 \equiv 348 \bmod 2 \equiv 0 \\
H_2 &\equiv (97 \cdot 5) \bmod 3 \equiv 485 \bmod 3 \equiv 2 \\
H_1 &\equiv (99 \cdot 7) \bmod 5 \equiv 693 \bmod 5 \equiv 3
\end{aligned}$$

So the resulting hash would be 320.

3.3 One-wayness and Discrete Algorithms

Hash functions are built to be one-way, easy to compute but hard to reverse. This is modeled after number-theoretic problems like the discrete logarithm problem (DLP):

$$\text{Given } a^{x_i} \bmod p = H_i, \text{ find } x$$

Even with full knowledge of a , b , and p , this is computationally hard if p is large. This difficulty inspires one-way constructions in hashing, where reversing a hash (to get the original password) is computationally infeasible [3].

Example 3. Suppose we hash "dog". Let's use the DLP for each letter with parameters $a = 5$ and $p = 97$, where x is the ASCII value of the letter. This time, we write the ASCII value back to its corresponding alphanumeric equivalent. Then we have,

$$\begin{aligned}
H_1 &\equiv 5^{100} \bmod 97 \equiv 43 = "+" \\
H_2 &\equiv 5^{111} \bmod 97 \equiv 46 = "." \\
H_3 &\equiv 5^{103} \bmod 97 \equiv 40 = "("
\end{aligned}$$

Thus, the hash is "+.(".

4 Real-World Hash Algorithms

Password Hashing contributes to a larger ecosystem of data with regards to ensuring data confidentiality, integrity, and security. With the help of mathematical concepts driven from number theory, modern cryptographic systems have evolved through time, further providing various techniques that offer stronger defense against attack vectors. Further looking into these, hashing is a one-way mathematical function that turns data into a string of nondescript text that cannot be reversed or decoded[9]. On the other hand, it is a way of scrambling data so that only authorized parties can understand the information[10]. In comparison, a key difference between the two is that hashing involves a one-way process while encryption, a two-way process. With regards to the contribution of number theory to these techniques, hashing often involves modular arithmetic, bit-wise operations, and prime constants, while encryption relies more directly on algebraic structures like finite fields, modular exponentiation, and public-private key pairs.

Especially with the evolution of technology, techniques for password hashing has multiplied. To further examine password hashing in a real-world setting, we take the three common techniques used: SHA-256, Bcrypt, and Argon2. These algorithms implement number-theoretic concepts to achieve critical goals such as unpredictability, irreversibility, and resistance to attack. Although all three techniques aim to provide the same processes, their implementation, properties and characteristics differ.

Starting with SHA-256, this technique is part of the SHA-2 algorithm, where SHA stands for Secure Hash Algorithm [12]. The origins of SHA-256 can be traced back to

Hashing	Encryption
• One-way process; cannot be reversed • Converts data into a fixed-size string • Used for: – verifying data integrity – detecting tampering – securely storing passwords • Algorithms include: SHA-256, Bcrypt, Argon	• Converts data into an unreadable format • Can only be accessed with a decryption key • Protects data in transit and at rest • Common methods include: – symmetric (e.g. AES) – asymmetric (e.g. RSA) – block and stream ciphers

Table 1: Hashing vs Encryption

the National Security Agency (NSA) of the USA, where engineers at the agency invested heavily in developing this algorithm, and it was first published in 2001 [13]. In correlation to the ever evolving science of hashing, the primary objective of SHA-256 was to optimize earlier hashing functions, posing an interest in strengthening data security. Its process involves generating a 256-bit hexadecimal string through a series of modular additions, bitwise operations, and constants derived from prime numbers. In secure password storage, systems store the SHA-256 hash of the password instead of storing the actual password. This enhances security by preventing exposure to plain-text passwords in case of a data breach [12]. In essence, SHA-256 is a secure and widely-used algorithm ideal for blockchain, SSL certs and data storage.

Bcrypt, another password hashing technique, uses modular arithmetic and key stretching in its processes. As defined, it is an adaptive password hashing function designed to secure passwords by transforming them into unreadable hashes[14]. This technique was created by Niels Provos and David Mazières, using the Blowfish cipher algorithm as a basis [15]. Its process includes complex hashing, where a user’s password is transformed into a fixed-length thread of characters. Since this technique includes built-in salting and adapts to computational advances, it is considered a go-to choice for password storage, also providing a strong security.

Lastly, Argon2 is a memory-hard function for password hashing. This further offers an advanced customization and enhanced security, resisting GPU and ASIC attacks. In its emergence, Argon2, which is a key derivation function (KDF) designed to defend systems against attacks, was developed by Alex Biryukov and a team of experts. This technique won the 2015 Password Hashing Competition (PHC) [16]. Argon2 is designed to have both configurable runtime as well as memory consumption. This technique has three variants, namely, Argon2d, Argon2i, and Argon2id[17]. Moreover, taking the comparing Argon 2’d and Argon21, Argon2d best protects data against time-memory trade-offs,

while Argon2i against side-channel attacks.

5 Summary and Conclusion

Generally, mathematics is used every day in different applications, in different fields. As most computer systems use passwords to ensure data security, forcing their users to create complex passwords, users often reuse the same password across multiple accounts. This raises the risk if one of them is compromised. With the help of mathematical foundational concepts, these issues are addressed with hashing techniques, further providing security and confidentiality. In essence, this paper explored the important role number theory plays in the effectiveness of password hashing algorithms.

The foundational principles of number theory, such as modular arithmetic, prime numbers, and congruences, provide the critical mathematical framework that underpins secure hashing functions. These concepts enable the design of algorithms that are computationally infeasible to reverse or collide, thereby protecting sensitive password information from unauthorized access. Furthermore, the continuous development of cryptographic primitives inspired by number theory ensures that password hashing methods evolve to counteract emerging threats and advances in attack strategies.

Overall, the application of number theory in password security demonstrates the critical connection between mathematics and practical cybersecurity solutions. This connection strengthens data protection and highlights the importance of mathematics in addressing real-world technological needs.

References

- [1] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed., Chapman and Hall/CRC, 2006.
- [2] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, 2020. Available online at: <https://crypto.stanford.edu/~dabo/cryptobook/>
- [3] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
- [4] N. Dotsenko, I. Chumachenko, A. Galkin, H. Kuchuk, and D. Chumachenko, *Modeling the Transformation of Configuration Management Processes in a Multi-Project Environment*, Sustainability, MDPI, 2023. DOI: <https://doi.org/10.3390/su151914308>
- [5] F. E. De Guzman, B. D. Gerardo, and R. P. Medina, *Implementation of Enhanced Secure Hash Algorithm Towards a Secured Web Portal*, in *2019 IEEE 4th International Conference on Computer and Communication Systems (ICCCS)*, Singapore, 2019, pp. [page numbers if available]. DOI: [10.1109/CCOMS.2019.8821763](https://doi.org/10.1109/CCOMS.2019.8821763)
- [6] V. Fedorchenko, O. Yeroshenko, O. Shmatko, O. Kolomiitsev, and M. Omarov, *Methods of Information Systems Protection*, in *[Journal or Conference Name if available]*, vol. 4, 2024, pp. [pages if available]. DOI: <https://doi.org/10.20998/2522-9052.2024.4.11>

- [7] M. Kamal, M. Abdu, and M. Hossain, *Modern Cryptography: Theory and Practice*, 1st ed., New York: Wiley, 2019, 432 pp. DOI: <https://doi.org/10.1002/9781119471227>
- [8] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, 1st ed., Boca Raton: CRC Press, 1997, 810 pp. DOI: <https://doi.org/10.1201/9780429466335>
- [9] Vaideeswaran, N. (2024, January 16). *Hashing in Cybersecurity*. Crowdstrike. <https://www.crowdstrike.com/en-us/cybersecurity-101/data-protection/data-hashing/>
- [10] Cloudflare. (n.d.). *What is encryption?*. <https://www.cloudflare.com/learning/ssl/what-is-encryption/>
- [11] Hughes, A. (2024, December 19). *Encryption vs. Hashing vs. Salting - What's the Difference?*. <https://www.pingidentity.com/en/resources/blog/post/encryption-vs-hashing-vs-salting.html>
- [12] Encryption consulting. (2024, March 4). *What is SHA-256*. <https://www.encryptionconsulting.com/education-center/sha-256/>
- [13] Supra. (2024, March 25). *The NSA and Bitcoin: Origins of the SHA-256 Hashing Algorithm*. <http://supra.com/academy/the-nsa-and-bitcoin-origins-of-the-sha-256-hashing-algorithm/>
- [14] Grigutyte, M. (2023, June 16). *What is bcrypt and how does it work?*. <https://nordvpn.com/blog/what-is-bcrypt/>
- [15] jumpcloud. (2025, April 22). *What is Bcrypt?*. <https://jumpcloud.com/it-index/what-is-bcrypt>
- [16] jumpcloud. (2025, April 22). *What is Argon2?*. <https://jumpcloud.com/it-index/what-is-argon2>
- [17] What is Argon2? (n.d.). <https://argon2-cffi.readthedocs.io/en/stable/argon2.html>